

EDA: Detecção de Intrusão em Redes UAV

Desafio: Um sistema de drones de precisão agrícola opera em enxame (swarm) trocando dados de telemetria e comandos via rede FANET. Ataques de rede comprometem a integridade das operações, podendo derrubar drones, desviar rotas ou causar colisões. O objetivo é construir um classificador que, a partir de métricas de fluxo de rede, identifique em tempo real se um fluxo é tráfego normal ou um dos quatro ataques conhecidos: **Blackhole, Flooding, Sybil ou Wormhole**.

Dataset: UAVIDS-2025 **URL:** <https://dx.doi.org/10.21227/j5p4-zt27>

Tipos de Ataques em Redes FANET

Blackhole Attack



No description has been provided for this image

O Ataque Blackhole ocorre quando um nó malicioso anuncia falsamente possuir a melhor rota para um destino, atraindo todo o tráfego da rede para si. Em vez de encaminhar os pacotes, o nó simplesmente os descarta, engolindo o tráfego como um buraco negro. Em redes de drones, isso pode silenciar telemetria crítica ou comandos de controle e comprometer a missão inteira.

Flooding Attack



No description has been provided for this image

O Ataque Flooding consiste em inundar a rede com um volume massivo de pacotes inúteis ou requisições falsas, sobrecarregando CPU, memória e largura de banda dos nós. Em enxames de drones, o flooding pode esgotar a bateria dos nós e deixar os canais de comunicação indisponíveis, causando perda de controle sobre os veículos.

Sybil Attack



No description has been provided for this image

No Ataque Sybil, um único nó malicioso cria várias identidades falsas na rede e se faz passar por diversos dispositivos legítimos. Com isso ele consegue manipular protocolos de votação, roteamento e consenso. Em redes de drones, o atacante pode influenciar decisões coletivas do enxame, forjando a presença de nós que não existem e desviando rotas de forma coordenada.

Wormhole Attack



No description has been provided for this image

O Ataque Wormhole envolve dois nós maliciosos ligados por um tunel secreto fora da rede legítima. Um nó captura pacotes em um ponto e os retransmite na hora para o complice em outro ponto distante, criando a ilusão de um link de baixa latência. Isso engana os protocolos de roteamento, que passam a preferir essa rota artificial, permitindo interceptar e manipular o tráfego.

Setup

```
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score, StratifiedKFold

# Configuração visual padrão
sns.set_theme(style='whitegrid', palette='Set2')
plt.rcParams['figure.dpi'] = 120
plt.rcParams['savefig.bbox'] = 'tight'

RANDOM_STATE = 42
DATASET_PATH = 'data/raw/UAVIDS-2025.csv'

# Paletas
PALETTE = {
    'Normal Traffic': '#2196F3',    # azul
    'Blackhole Attack': '#F44336',  # vermelho
    'Flooding Attack': '#FF9800',   # laranja
    'Sybil Attack': '#9C27B0',      # roxo
    'Wormhole Attack': '#E91E63',   # rosa
}
```

1. Carregamento e Inspeção Inicial

```
df = pd.read_csv(DATASET_PATH)

print(f'Dimensões: {df.shape[0]:,} linhas × {df.shape[1]} colunas')
print()
df.info()
```

Dimensões: 122,171 linhas × 23 colunas

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 122171 entries, 0 to 122170

Data columns (total 23 columns):

#	Column	Non-Null Count	Dtype
0	FlowID	122171 non-null	int64
1	FlowDuration/s	122171 non-null	float64
2	SrcAddr	122171 non-null	object
3	SrcPort	122171 non-null	int64
4	DstAddr	122171 non-null	object
5	DstPort	122171 non-null	int64
6	Protocol	122171 non-null	object
7	TxPackets	122171 non-null	int64
8	RxPackets	122171 non-null	int64
9	LostPackets	122171 non-null	int64
10	TxBytes	122171 non-null	int64
11	RxBytes	122171 non-null	int64
12	TxPacketRate/s	122171 non-null	float64
13	RxPacketRate/s	122171 non-null	float64
14	TxByteRate/s	122171 non-null	float64
15	RxByteRate/s	122171 non-null	float64
16	MeanDelay/s	122171 non-null	float64
17	MeanJitter/s	122171 non-null	float64
18	Throughput/Kbps	122171 non-null	float64
19	MeanPacketSize	122171 non-null	int64
20	PacketDropRate	122171 non-null	float64
21	AverageHopCount	122171 non-null	float64
22	label	122171 non-null	object

dtypes: float64(10), int64(9), object(4)

memory usage: 21.4+ MB

Sumariza as estatísticas descritivas de todas as colunas com `df.describe(include='all').T`. O transposto facilita a leitura: mostra count, media, desvio, minimo e maximo para as numericas, e count, unique, top e freq para as categoricas. Ajuda a perceber escalas muito diferentes entre as features e valores fora do dominio esperado.

```
df.describe(include='all').T
```

	count	unique	top	freq	mean	std	min	
FlowID	122171.0	NaN	NaN	NaN	61086.0	35267.874206	1.0	305
FlowDuration/s	122171.0	NaN	NaN	NaN	696.714475	780.052068	0.000076	124
SrcAddr	122171	176	192.168.0.5	3341	NaN	NaN	NaN	
SrcPort	122171.0	NaN	NaN	NaN	169.137348	278.648982	9.0	
DstAddr	122171	218	192.168.0.24	1929	NaN	NaN	NaN	
DstPort	122171.0	NaN	NaN	NaN	169.137348	278.648982	9.0	
Protocol	122171	1	UDP	122171	NaN	NaN	NaN	
TxPackets	122171.0	NaN	NaN	NaN	486.930368	1933.094801	1.0	
RxPackets	122171.0	NaN	NaN	NaN	426.070737	1367.693106	0.0	
LostPackets	122171.0	NaN	NaN	NaN	60.436159	1149.330904	0.0	
TxBytes	122171.0	NaN	NaN	NaN	35872.445629	124664.421944	30.0	9
RxBytes	122171.0	NaN	NaN	NaN	31938.199311	102554.559542	0.0	3
TxPacketRate/s	122171.0	NaN	NaN	NaN	2.466915	130.044753	0.001197	0.04
RxPacketRate/s	122171.0	NaN	NaN	NaN	1.383054	58.987416	0.0	0.01
TxByteRate/s	122171.0	NaN	NaN	NaN	182.091598	9857.744667	0.044673	3.0
RxByteRate/s	122171.0	NaN	NaN	NaN	103.422636	4481.638475	0.0	1.08
MeanDelay/s	122171.0	NaN	NaN	NaN	0.207858	0.454055	0.0	0.01
MeanJitter/s	122171.0	NaN	NaN	NaN	0.14008	0.306921	0.0	0.00
Throughput/Kbps	122171.0	NaN	NaN	NaN	0.827381	35.85311	0.0	0.00
MeanPacketSize	122171.0	NaN	NaN	NaN	68.122885	13.838427	30.0	
PacketDropRate	122171.0	NaN	NaN	NaN	0.369328	0.309535	0.0	0.09
AverageHopCount	122171.0	NaN	NaN	NaN	0.264616	0.653093	0.0	
label	122171	5	Normal Traffic	26172	NaN	NaN	NaN	

Mostra as primeiras linhas com `df.head()` para uma inspecao rapida do formato e do conteudo, conferindo se os tipos estao corretos e se a estrutura do DataFrame e a que esperamos.

```
df.head()
```

	FlowID	FlowDuration/s	SrcAddr	SrcPort	DstAddr	DstPort	Protocol	TxPackets	RxPackets
0	1	175.007	192.168.0.150	9	192.168.0.16	9	UDP	251	21
1	2	175.008	192.168.0.150	9	192.168.0.19	9	UDP	251	22
2	3	175.010	192.168.0.150	9	192.168.0.24	9	UDP	251	22
3	4	175.014	192.168.0.150	9	192.168.0.34	9	UDP	251	21
4	5	175.017	192.168.0.150	9	192.168.0.39	9	UDP	251	21

5 rows × 23 columns

Lista o tipo de cada coluna com `df.dtypes`. Tipos errados, como um numero guardado como `object`, precisam ser corrigidos antes da modelagem.

```
print(df.dtypes.to_string())
```

```
FlowID                int64
FlowDuration/s        float64
SrcAddr               object
SrcPort               int64
DstAddr               object
DstPort               int64
Protocol              object
TxPackets             int64
RxPackets             int64
LostPackets           int64
TxBytes               int64
RxBytes               int64
TxPacketRate/s        float64
RxPacketRate/s        float64
TxByteRate/s          float64
RxByteRate/s          float64
MeanDelay/s           float64
MeanJitter/s          float64
Throughput/Kbps        float64
MeanPacketSize        int64
PacketDropRate         float64
AverageHopCount        float64
label                 object
```

O que a inspecao inicial mostrou:

- `FlowID` e um inteiro sequencial, apenas identificador. Vai ser removido antes da modelagem.
- `SrcAddr` e `DstAddr` sao enderecos IP e parecem ter cardinalidade alta.
- As features numericas de volume estao numa escala muito maior que as de performance, entao o `StandardScaler` vai ser necessario para padronizar.

- Não apareceu nenhum erro de tipo, o que faz sentido porque o dataset foi gerado por simulação controlada.

2. Distribuição da Variável-Alvo

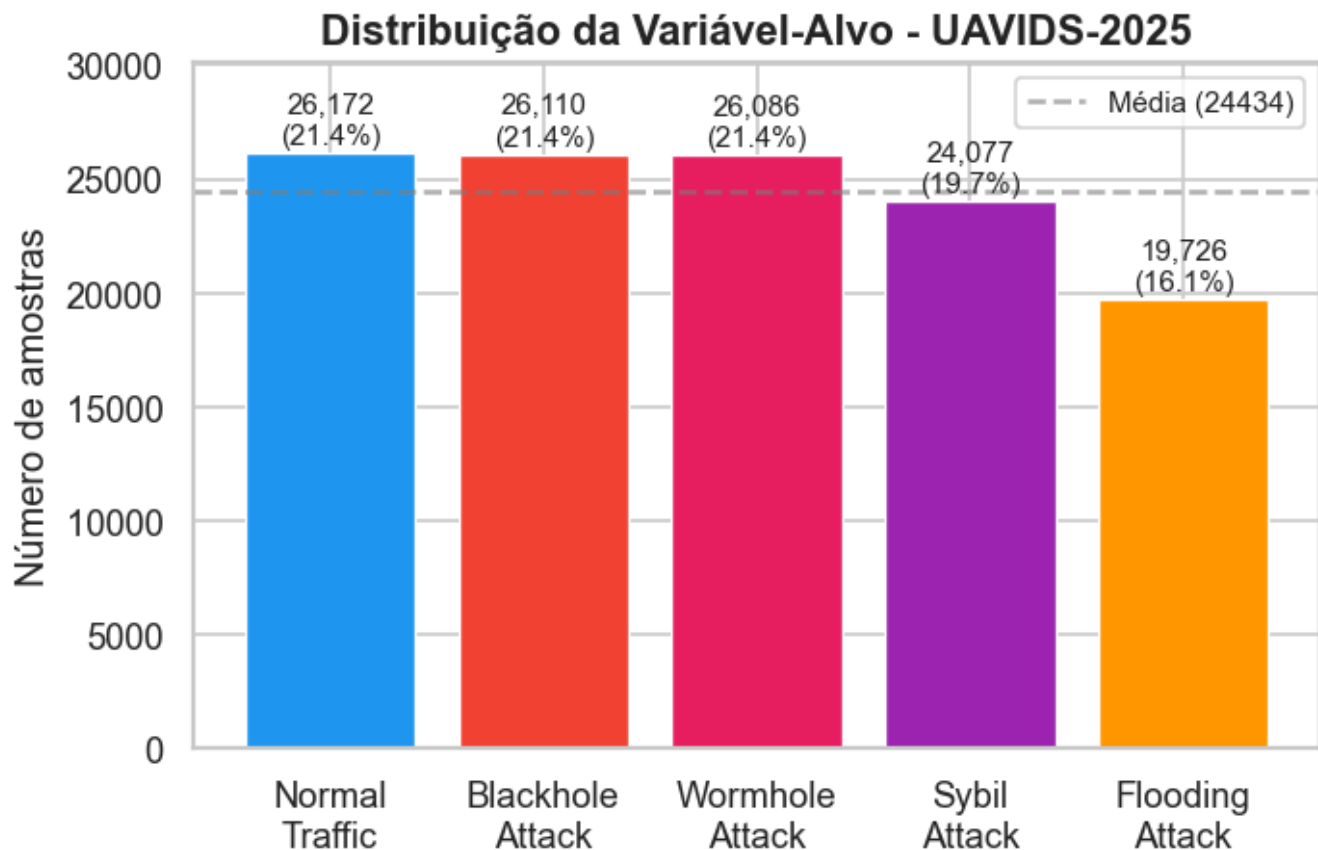
```
contagem = df['label'].value_counts()
percentual = df['label'].value_counts(normalize=True).mul(100).round(1)

ordem = list(contagem.index)

fig, ax = plt.subplots(figsize=(6, 4))

bars = ax.bar(
    [c.replace(' ', '\n') for c in ordem],
    [contagem[c] for c in ordem],
    color=[PALETTE[c] for c in ordem],
    edgecolor='white', linewidth=0.8
)
for bar, cls in zip(bars, ordem):
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 150,
        f'{contagem[cls]:,}\n({percentual[cls]}%)',
        ha='center', va='bottom', fontsize=9
    )
ax.set_title('Distribuição da Variável-Alvo - UVAIDS-2025', fontsize=13, fontweight='bold')
ax.set_ylabel('Número de amostras')
ax.set_ylim(0, contagem.max() * 1.15)
ax.axhline(contagem.mean(), color='gray', linestyle='--', alpha=0.6, label=f'Média ({contagem.mean():.2f})')
ax.legend(fontsize=9)

plt.tight_layout()
plt.show()
```



O que o grafico da variavel-alvo mostra:

- Normal Traffic 26.172 (21,4%), Blackhole 26.110 (21,4%), Wormhole 26.086 (21,4%), Sybil 24.077 (19,7%) e Flooding 19.726 (16,1%).
- Tres classes ficam praticamente empatadas em 21,4% e o Flooding fica cerca de 5 pontos percentuais abaixo.

3. Distribuições das Features Numéricas

```
# Definindo grupos de features por categoria
CONNECTION_FEATURES = ['FlowDuration/s', 'SrcPort', 'DstPort']
VOLUME_FEATURES = [
    'TxPackets', 'RxPackets', 'LostPackets',
    'TxBytes', 'RxBytes',
    'TxPacketRate/s', 'RxPacketRate/s',
    'TxByteRate/s', 'RxByteRate/s',
    'MeanPacketSize'
]
PERFORMANCE_FEATURES = [
    'MeanDelay/s', 'MeanJitter/s',
    'Throughput/Kbps', 'PacketDropRate', 'AverageHopCount'
]

NUM_FEATURES = CONNECTION_FEATURES + VOLUME_FEATURES + PERFORMANCE_FEATURES
CAT_FEATURES = ['Protocol'] # SrcAddr e DstAddr analisados separadamente
```

```
print(f'Features numéricas: {len(NUM_FEATURES)}')
print(f'Features categóricas (úteis): {len(CAT_FEATURES)}')
print(f'Identificadores a remover: FlowID, SrcAddr, DstAddr')
```

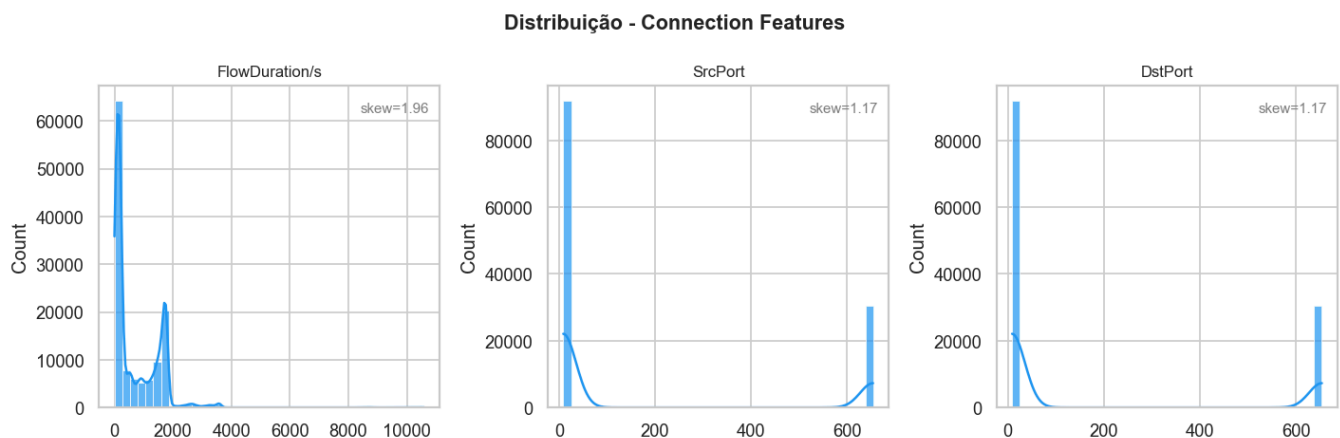
Features numéricas: 18
 Features categóricas (úteis): 1
 Identificadores a remover: FlowID, SrcAddr, DstAddr

Inspeciona a distribuicao das features de conexao (duracao do fluxo e portas). O KDE sobreposto ajuda a enxergar bimodalidade e assimetria que as barras sozinhas escondem. As portas sao numericas mas se comportam como categoricas, entao o histograma mostra se ha concentracao em faixas especificas.

```
# Histogramas - Connection Features
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
fig.suptitle('Distribuição - Connection Features', fontsize=13, fontweight='bold')

for ax, feat in zip(axes, CONNECTION_FEATURES):
    sns.histplot(df[feat], bins=40, kde=True, ax=ax, color='#2196F3', alpha=0.7)
    ax.set_title(feat, fontsize=10)
    ax.set_xlabel('')
    skew = df[feat].skew()
    ax.text(0.97, 0.95, f'skew={skew:.2f}', transform=ax.transAxes,
           ha='right', va='top', fontsize=9, color='gray')

plt.tight_layout()
plt.show()
```



O que o grafico das Connection Features mostra:

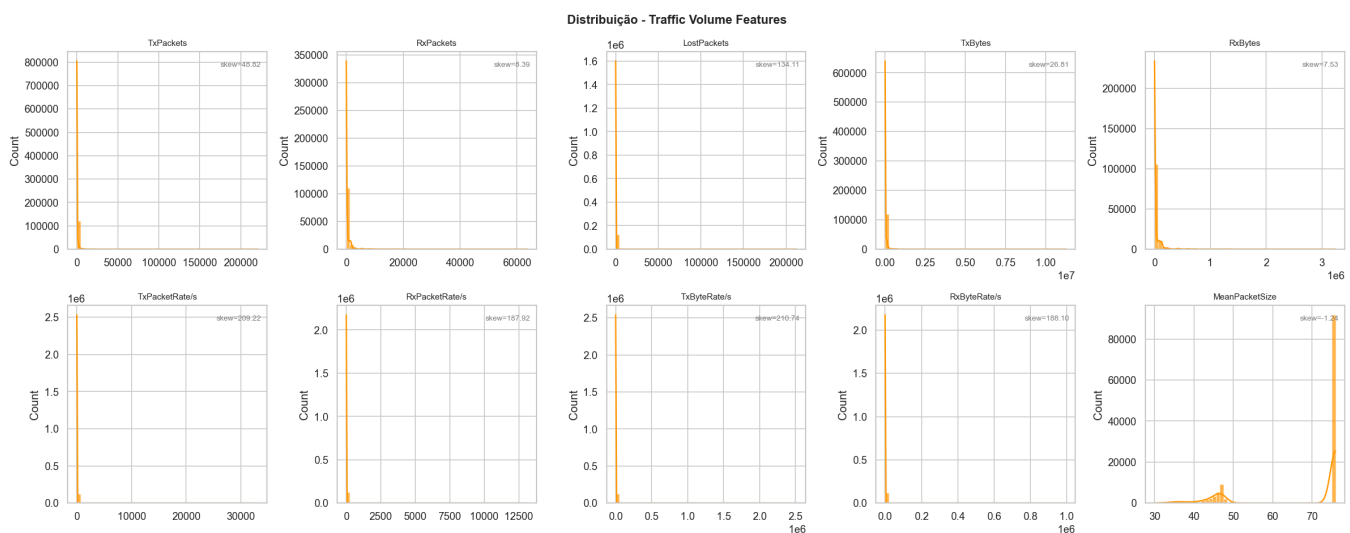
- FlowDuration/s tem assimetria de 1,96 e cauda direita forte. O pico principal fica em fluxos curtos, ate cerca de 200s, com mais de 60 mil amostras, e ha um segundo pico perto de 1.800 a 2.000s, sugerindo dois regimes de trafego (sessoes curtas e prolongadas).
- SrcPort e DstPort tem assimetria de 1,17 e distribuicao bimodal identica entre as duas, com dois clusters dominantes.

Inspeciona a escala e a assimetria das features de volume de trafego com `sns.histplot(kde=True)` numa grade 2x5. Como features de contagem (TxPackets, RxBytes) costumam ser bem assimetricas, confirmar isso aqui justifica usar `StandardScaler` no pipeline em vez de `MinMaxScaler`, que e mais sensivel a valores extremos.

```
# Histogramas - Traffic Volume Features
fig, axes = plt.subplots(2, 5, figsize=(20, 8))
fig.suptitle('Distribuição - Traffic Volume Features', fontsize=13, fontweight='bold')
axes = axes.flatten()

for ax, feat in zip(axes, VOLUME_FEATURES):
    sns.histplot(df[feat], bins=50, kde=True, ax=ax, color='#FF9800', alpha=0.7)
    ax.set_title(feat, fontsize=9)
    ax.set_xlabel('')
    skew = df[feat].skew()
    ax.text(0.97, 0.95, f'skew={skew:.2f}', transform=ax.transAxes,
           ha='right', va='top', fontsize=8, color='gray')

plt.tight_layout()
plt.show()
```



O que o grafico das Traffic Volume Features mostra:

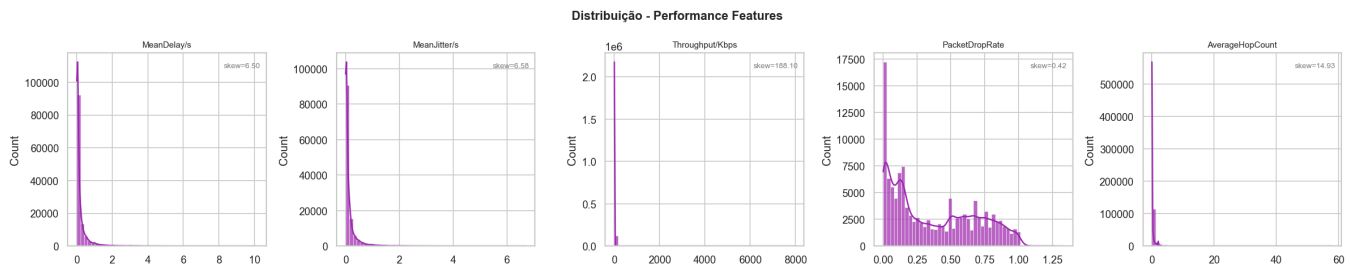
- As 10 features de volume tem assimetria muito alta (TxPackets ~88, RxPackets ~8, LostPackets ~130, TxBytes ~26, RxBytes ~7, TxPacketRate/s acima de 200, RxPacketRate/s ~187, TxByteRate/s ~210, RxByteRate/s ~180, MeanPacketSize ~1,2).
- A distribuicao se concentra numa unica barra perto de zero com cauda muito longa, (Esse é o padrao de trafego de rede onde poucos fluxos carregam grandes volumes).
- MeanPacketSize e a unica feature aproximadamente normal (assimetria ~1,2), com dois modos visiveis perto de 30 e 50 bytes.

```
# Histogramas - Performance Features
fig, axes = plt.subplots(1, 5, figsize=(20, 4))
fig.suptitle('Distribuição - Performance Features', fontsize=13, fontweight='bold')

for ax, feat in zip(axes, PERFORMANCE_FEATURES):
    sns.histplot(df[feat], bins=50, kde=True, ax=ax, color='#9C27B0', alpha=0.7)
    ax.set_title(feat, fontsize=9)
    ax.set_xlabel('')
    skew = df[feat].skew()
    ax.text(0.97, 0.95, f'skew={skew:.2f}', transform=ax.transAxes,
```

```
ha='right', va='top', fontsize=8, color='gray')
```

```
plt.tight_layout()
plt.show()
```



O que o gráfico das Performance Features mostra:

- MeanDelay/s (assimetria 6,5) e MeanJitter/s (assimetria 6,58) tem caudas longas, mas com a massa concentrada perto de zero. Valores acima de 4s são raros e provavelmente ligados a ataques.
- Throughput/Kbps (assimetria 188) tem quase toda a massa em zero, com muitas amostras no primeiro bin e uma cauda enorme. Poucos fluxos atingem throughput alto, o que é a provável assinatura do Flooding.
- PacketDropRate (assimetria 0,42) é a única feature aproximadamente uniforme entre 0 e 1, com pico no zero. É o sinal mais limpo do dataset.
- AverageHopCount (assimetria 14,9).

Observações gerais das distribuições numéricas:

- 14 das 15 features numéricas tem assimetria alta, então a padronização com `StandardScaler` é obrigatória dentro do Pipeline.
- PacketDropRate precisa de uma verificação extra: existem 81 registros com valor maior que 1.
- Throughput/Kbps, PacketDropRate e AverageHopCount são bons candidatos para distinguir Flooding, Blackhole e Wormhole.

4. Distribuições das Features Categóricas

```
for col in ['Protocol', 'SrcAddr', 'DstAddr']:
    n_unique = df[col].nunique()
    print(f'{col}: {n_unique:,} valores únicos')

print()
print('Protocol - value_counts:')
vc = df['Protocol'].value_counts()
vc_pct = df['Protocol'].value_counts(normalize=True).mul(100).round(2)
print(pd.DataFrame({'Contagem': vc, '%': vc_pct}).to_string())
```

Protocol: 1 valores únicos
SrcAddr: 176 valores únicos
DstAddr: 218 valores únicos

Protocol - value_counts:
Contagem %
Protocol
UDP 122171 100.0

Mostra a distribuicao das 3 features categoricas com `sns.countplot` (Protocol e o top-15 de SrcAddr e DstAddr) para decidir se entram ou saem da modelagem. Os countplots revelam a cardinalidade real e as categorias dominantes. Para os IPs, com muitos valores unicos, o top-15 evita um grafico ilegivel e ja mostra se ha concentracao ou dispersao.

```
# Countplot das features categoricas
fig, axes = plt.subplots(1, 3, figsize=(20, 5))
fig.suptitle('Distribuicao das Features Categoricas', fontsize=14, fontweight='bold')

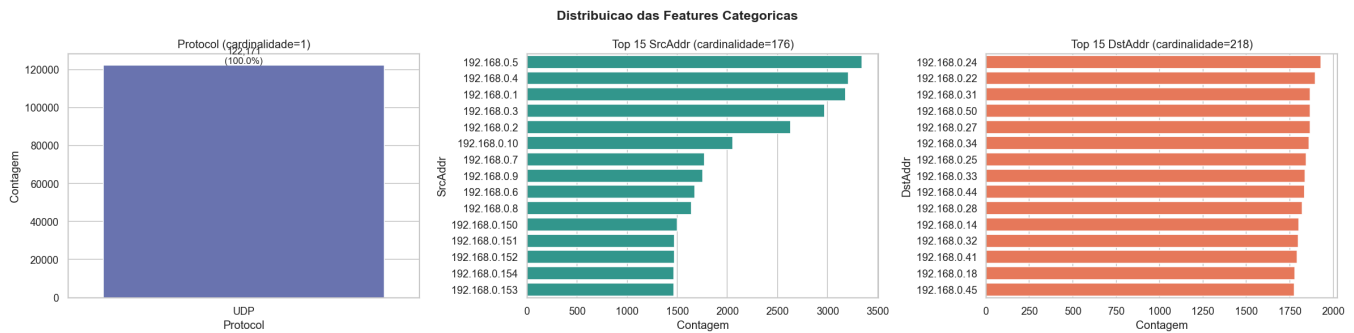
# Protocol: cardinalidade=1, countplot direto
sns.countplot(data=df, x='Protocol', ax=axes[0], color='#5C6BC0')
axes[0].set_title(f'Protocol (cardinalidade={df["Protocol"].nunique()})')
axes[0].set_ylabel('Contagem')
for p in axes[0].patches:
    h = p.get_height()
    axes[0].annotate(f'{int(h):,}\n({h/len(df)*100:.1f}%)',
                    (p.get_x() + p.get_width()/2., h),
                    ha='center', va='bottom', fontsize=10)

# SrcAddr: cardinalidade alta, top 15
top_src = df['SrcAddr'].value_counts().head(15)
sns.barplot(x=top_src.values, y=top_src.index, ax=axes[1], color='#26A69A', orient='h')
axes[1].set_title(f'Top 15 SrcAddr (cardinalidade={df["SrcAddr"].nunique()})')
axes[1].set_xlabel('Contagem')

# DstAddr: cardinalidade alta, top 15
top_dst = df['DstAddr'].value_counts().head(15)
sns.barplot(x=top_dst.values, y=top_dst.index, ax=axes[2], color='#FF7043', orient='h')
axes[2].set_title(f'Top 15 DstAddr (cardinalidade={df["DstAddr"].nunique()})')
axes[2].set_xlabel('Contagem')

plt.tight_layout()
plt.savefig('img/eda_img/eda_categoricas.png', dpi=150, bbox_inches='tight')
plt.show()

print(f'\nProtocol: {df["Protocol"].nunique()} valor unico -> feature constante')
print(f'SrcAddr: {df["SrcAddr"].nunique()} IPs unicos -> cardinalidade alta')
print(f'DstAddr: {df["DstAddr"].nunique()} IPs unicos -> cardinalidade alta')
```



Protocol: 1 valor unico -> feature constante
 SrcAddr: 176 IPs unicos -> cardinalidade alta
 DstAddr: 218 IPs unicos -> cardinalidade alta

- **Protocol** : descartada. E 100% UDP nos 122.171 registros, ou seja, variancia zero e nenhum valor discriminativo. Um OneHotEncoding geraria uma coluna constante.
- **SrcAddr** e **DstAddr** : descartadas. Mesmo com cardinalidade moderada (176 e 218 IPs), sao enderecos de uma rede simulada fechada. Um modelo que aprende IPs especificos da simulacao NS-3 nao generaliza para redes reais, onde os IPs mudam a cada missao.
- **FlowID** : descartado. Identificador sequencial sem valor preditivo.
- Lista final de features para a modelagem: as 18 numericas, exceto FlowID (Protocol e os enderecos IP ficam de fora).

5. Relações Features x Alvo: Foco no Par Blackhole/Wormhole

```
# Boxplots das features mais discriminativas por classe – todas as 5 classes
features_key = ['PacketDropRate', 'Throughput/Kbps', 'AverageHopCount',
                'MeanDelay/s', 'MeanJitter/s', 'LostPackets']

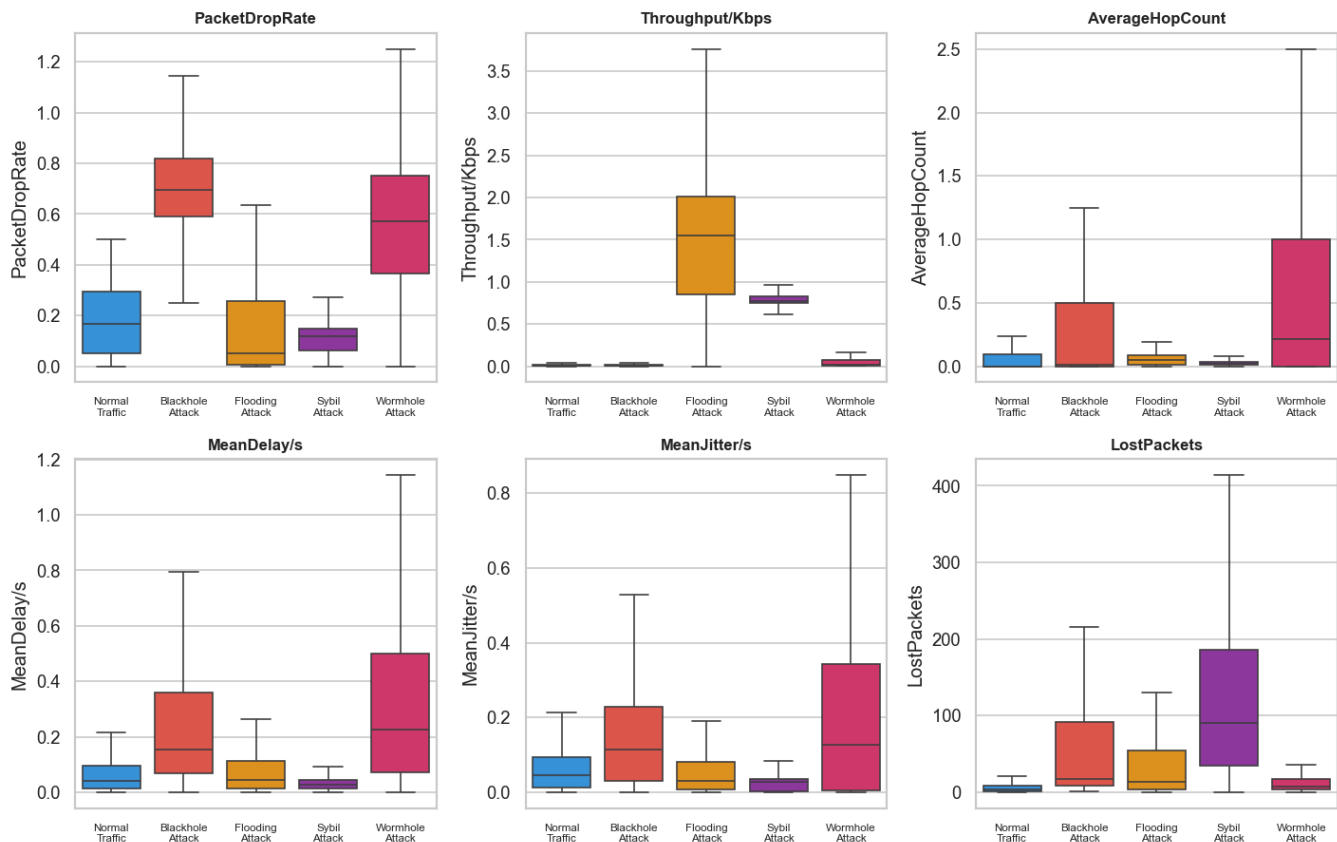
fig, axes = plt.subplots(2, 3, figsize=(12, 8))
fig.suptitle('Features de Performance por Classe - Assinaturas dos Ataques',
             fontsize=14, fontweight='bold')
axes = axes.flatten()

ordem_classes = list(PALETTE.keys())

for ax, feat in zip(axes, features_key):
    sns.boxplot(
        data=df, x='label', y=feat,
        order=ordem_classes,
        palette=PALETTE,
        ax=ax,
        showfliers=False # omitir outliers extremos para legibilidade
    )
    ax.set_title(feat, fontsize=10, fontweight='bold')
    ax.set_xlabel('')
    ax.set_xticklabels([c.replace(' ', '\n') for c in ordem_classes], fontsize=7)
```

```
plt.tight_layout()
plt.show()
```

Features de Performance por Classe - Assinaturas dos Ataques



O que os boxplots por classe mostram:

- PacketDropRate: o Blackhole tem a mediana mais alta (cerca de 0,70, com IQR de 0,59 a 0,82) e o Wormhole vem em segundo (cerca de 0,58, IQR de 0,37 a 0,76). Normal, Flooding e Sybil ficam abaixo de 0,20. É uma assinatura clara do Blackhole, que descarta pacotes ativamente.
- Throughput/Kbps: o Flooding domina com mediana de cerca de 1,55 (IQR de 0,84 a 2,00), o Sybil fica em torno de 0,80 e as outras três classes ficam perto de zero. Confirma o Flooding como gerador massivo de tráfego.
- AverageHopCount: o Wormhole tem distribuição bem mais larga (IQR de 0,01 a 1,01, máximo perto de 2,5) e o Blackhole vai até cerca de 1,25. As demais ficam próximas de zero. O Wormhole altera a topologia da rota, como esperado.
- MeanDelay/s e MeanJitter/s: o Wormhole tem as maiores medianas (delay de cerca de 0,22 e jitter de 0,14), seguido do Blackhole (delay 0,15 e jitter 0,12). Os túneis do Wormhole introduzem atraso e instabilidade.
- LostPackets: o Sybil tem a maior mediana em valor absoluto (cerca de 90), o Blackhole fica em torno de 25 e o Flooding em 20. Diferente do PacketDropRate, aqui é a contagem bruta, e ela revela o Sybil como o ataque que mais perde pacotes em volume.

Verifica a separabilidade usando duas features de desempenho, com um scatterplot colorido por classe numa amostra de 15 mil pontos. Escolhi o par MeanDelay e MeanJitter porque representam duas

dimensoes diferentes de degradacao: ataques que aumentam a latencia nao aumentam o jitter na mesma proporcao, o que tende a formar clusters visualmente distintos.

```
# Scatterplot MeanDelay x MeanJitter – par mais relevante para BH vs WH
# Subamostrar para visualização (100k pontos sobrepostos ficam ilegíveis)
df_sample = df.sample(n=min(15000, len(df)), random_state=RANDOM_STATE)

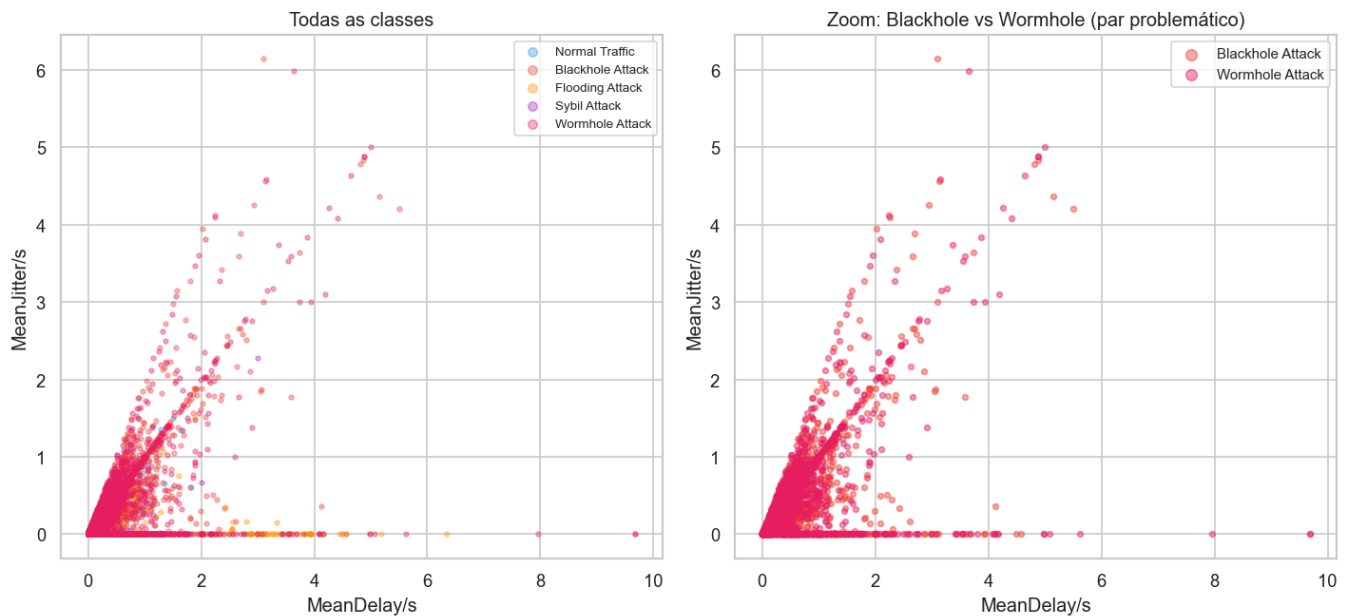
fig, axes = plt.subplots(1, 2, figsize=(12, 6))
fig.suptitle('Scatterplot MeanDelay x MeanJitter por Classe',
             fontsize=13, fontweight='bold')

# Painel esquerdo: todas as classes
for cls in ordem_classes:
    subset = df_sample[df_sample['label'] == cls]
    axes[0].scatter(
        subset['MeanDelay/s'], subset['MeanJitter/s'],
        c=PALETTE[cls], label=cls, alpha=0.35, s=8
    )
axes[0].set_xlabel('MeanDelay/s')
axes[0].set_ylabel('MeanJitter/s')
axes[0].set_title('Todas as classes')
axes[0].legend(fontsize=8, markerscale=2)

# Painel direito: apenas BH vs WH – o par problemático
bh_wh = df_sample[df_sample['label'].isin(['Blackhole Attack', 'Wormhole Attack'])]
for cls in ['Blackhole Attack', 'Wormhole Attack']:
    subset = bh_wh[bh_wh['label'] == cls]
    axes[1].scatter(
        subset['MeanDelay/s'], subset['MeanJitter/s'],
        c=PALETTE[cls], label=cls, alpha=0.45, s=12
    )
axes[1].set_xlabel('MeanDelay/s')
axes[1].set_ylabel('MeanJitter/s')
axes[1].set_title('Zoom: Blackhole vs Wormhole (par problemático)')
axes[1].legend(fontsize=9, markerscale=2)

plt.tight_layout()
plt.show()
```

Scatterplot MeanDelay x MeanJitter por Classe



O que o grafico de MeanDelay por MeanJitter mostra:

- No painel da esquerda, com as 5 classes, Normal e Flooding ficam aglomeradas no canto inferior esquerdo (delay e jitter perto de zero), e o Sybil tambem se concentra nessa regioao. Blackhole e Wormhole se espalham para cima e para a direita num padrao diagonal, chegando a delay de cerca de 10s e jitter de 6s.
- No painel da direita, com o zoom em Blackhole e Wormhole, as duas classes formam trilhas diagonais quase paralelas, com forte sobreposicao. O Wormhole alcanca valores extremos mais altos, mas nao da para separar as duas com esse par de features.
- Resumindo: delay e jitter separam bem os ataques lentos (Blackhole e Wormhole) do trafego comum, mas nao separam Blackhole de Wormhole entre si. Vamos precisar de outras features para esse par.

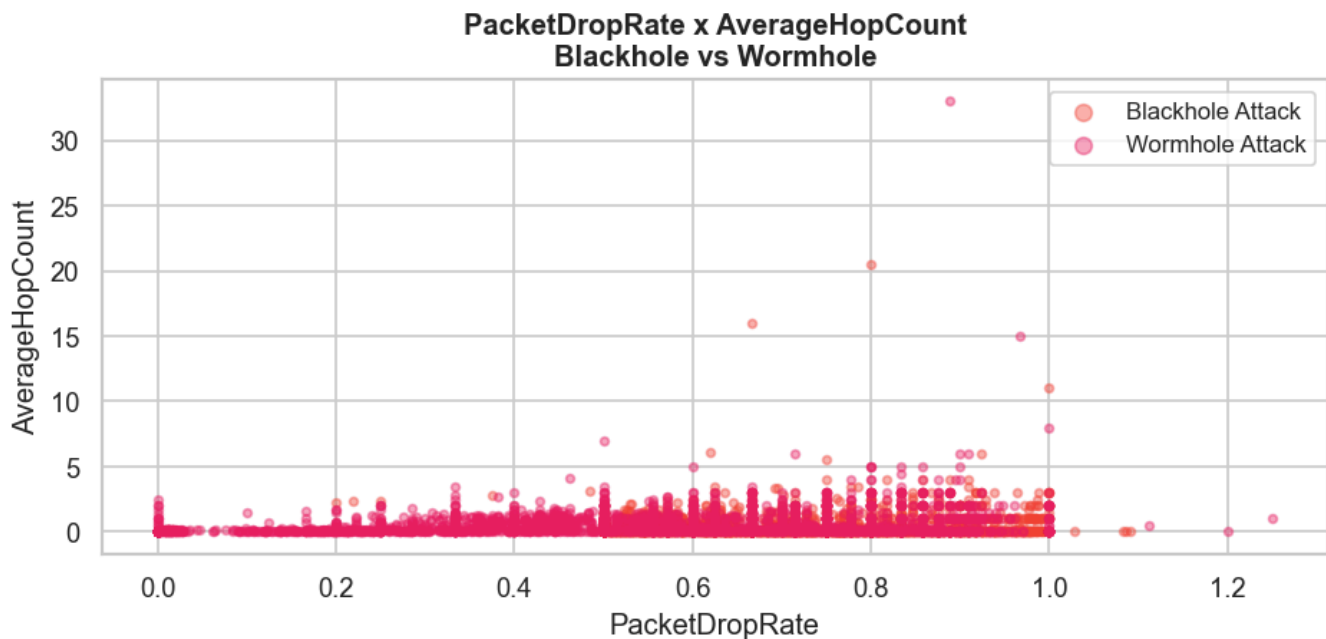
Analisa a assinatura que mais distingue Blackhole de Wormhole, o par mais dificil de separar do dataset. O scatterplot usa so essas duas classes (10 mil amostras). O Blackhole descarta pacotes ativamente, com PacketDropRate alto, enquanto o Wormhole cria um tunel artificial que aumenta o HopCount.

```
# PacketDropRate x AverageHopCount – assinatura primária BH vs WH segundo o artigo
fig, ax = plt.subplots(figsize=(8, 4))

bh_wh_full = df[df['label'].isin(['Blackhole Attack', 'Wormhole Attack'])].sample(
    n=min(10000, len(bh_wh)), random_state=RANDOM_STATE
)

for cls in ['Blackhole Attack', 'Wormhole Attack']:
    subset = bh_wh_full[bh_wh_full['label'] == cls]
    ax.scatter(
        subset['PacketDropRate'], subset['AverageHopCount'],
        c=PALETTE[cls], label=cls, alpha=0.4, s=12
    )
```

```
ax.set_xlabel('PacketDropRate')
ax.set_ylabel('AverageHopCount')
ax.set_title('PacketDropRate x AverageHopCount\nBlackhole vs Wormhole', fontweight='bold')
ax.legend(fontsize=10, markerscale=2)
plt.tight_layout()
plt.show()
```



O que o grafico de PacketDropRate por AverageHopCount (Blackhole e Wormhole) mostra:

- A maioria dos pontos das duas classes se concentra na faixa de AverageHopCount entre 0 e 2, espalhada por todo o eixo de PacketDropRate (0 a 1).
- Alguns outliers de Blackhole atingem HopCount alto (7, 16, 20,5) e um outlier de Wormhole chega a 33.
- A sobreposicao e grande.
- Resumindo: esse par de features sozinho nao resolve Blackhole versus Wormhole. Isso justifica criar features derivadas como `loss_ratio` e `tx_efficiency` para gerar mais separabilidade.

Observacoes gerais das relacoes entre features e alvo:

- PacketDropRate isola o Blackhole do resto, Throughput isola o Flooding e AverageHopCount captura o Wormhole, mas nenhuma feature sozinha resolve o par Blackhole versus Wormhole.
- O proximo passo e criar `loss_ratio = LostPackets / TxPackets` e `tx_efficiency = RxBytes / TxBytes` para ampliar a diferenca entre quem descarta pacotes (Blackhole) e quem reencaminha por tuneis (Wormhole).

6. Matriz de Correlação

```
corr_matrix = df[NUM_FEATURES].corr()

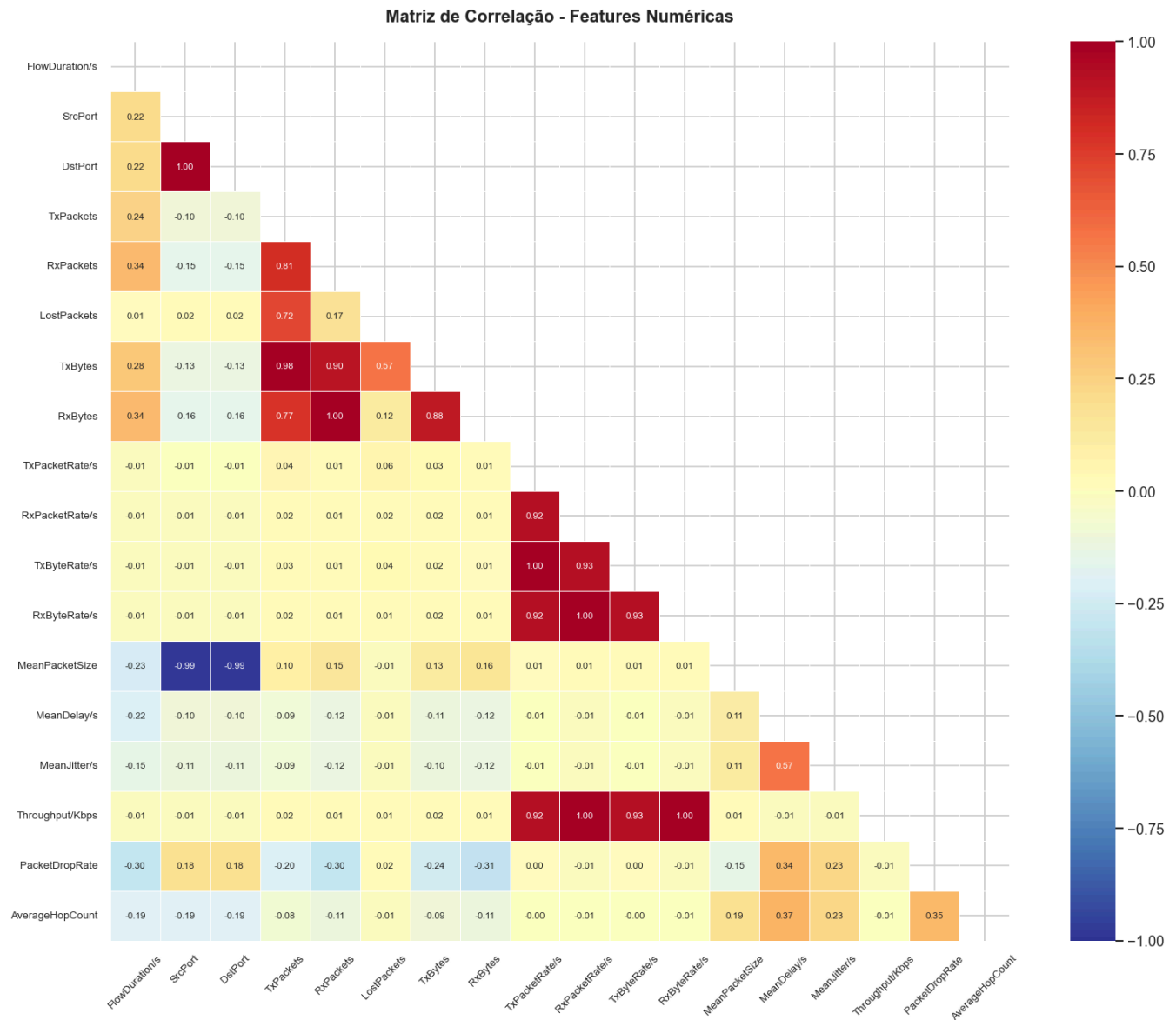
fig, ax = plt.subplots(figsize=(14, 11))
```

```

mask = np.triu(np.ones_like(corr_matrix, dtype=bool)) # mostrar só triângulo inferior

sns.heatmap(
    corr_matrix,
    mask=mask,
    annot=True, fmt='.2f',
    cmap='RdYlBu_r',
    center=0, vmin=-1, vmax=1,
    square=True,
    linewidths=0.3,
    annot_kws={'size': 7},
    ax=ax
)
ax.set_title('Matriz de Correlação - Features Numéricas',
             fontsize=13, fontweight='bold', pad=15)
ax.tick_params(axis='x', labelsiz=8, rotation=45)
ax.tick_params(axis='y', labelsiz=8, rotation=0)
plt.tight_layout()
plt.show()

```



O que a matriz de correlacao de Pearson mostra:

- Pares quase perfeitamente correlacionados (r acima de 0,9): Throughput/Kbps com TxByteRate/s (perto de 1,00), TxPackets com RxPackets (cerca de 0,98), TxByteRate/s com RxByteRate/s (0,92), TxPacketRate/s com RxPacketRate/s (0,92) e MeanPacketSize com TxBytes (0,77).
- Correlacoes negativas que chamam atencao: FlowDuration/s com TxByteRate, RxByteRate e Throughput, em torno de -0,30. Fluxos mais longos tem taxas mais baixas, o que faz sentido fisicamente.
- PacketDropRate com AverageHopCount tem correlacao positiva fraca (cerca de +0,31). E o sinal estatistico que motiva combinar as duas no feature engineering.
- As demais correlacoes ficam abaixo de 0,20 em modulo, indicando que as features sao em sua maioria independentes.
- Resumindo: ha multicolinearidade forte dentro do bloco de volume e taxa, mas como modelos como Random Forest e XGBoost lidam bem com redundancia, entao nao vamos remover features manualmente.

Identifica a multicolinearidade entre as features filtrando o `df.corr()` pelo limiar de 0,7 em modulo. A correlacao de Pearson e adequada para features continuas. Os pares muito correlacionados poderiam ser removidos, mas a decisao foi mante-los e deixar o modelo lidar com a redundancia, ja que o StandardScaler e a regularizacao dos modelos de arvore dao conta disso.

```
# Listar pares com |r| >= 0.7 (multicolinearidade)
threshold = 0.7
high_corr = []

for i in range(len(corr_matrix.columns)):
    for j in range(i + 1, len(corr_matrix.columns)):
        r = corr_matrix.iloc[i, j]
        if abs(r) >= threshold:
            high_corr.append({
                'Feature A': corr_matrix.columns[i],
                'Feature B': corr_matrix.columns[j],
                'r': round(r, 3)
            })

hc_df = pd.DataFrame(high_corr).sort_values('r', key=abs, ascending=False)
print(f'Pares com |r| ≥ {threshold}: {len(hc_df)}')
print(hc_df.to_string(index=False))
```

Pares com $|r| \geq 0.7$: 20

Feature A	Feature B	r
SrcPort	DstPort	1.000
RxPacketRate/s	RxByteRate/s	1.000
RxPacketRate/s	Throughput/Kbps	1.000
RxByteRate/s	Throughput/Kbps	1.000
TxPacketRate/s	TxByteRate/s	0.999
RxPackets	RxBytes	0.997
SrcPort	MeanPacketSize	-0.990
DstPort	MeanPacketSize	-0.990
TxPackets	TxBytes	0.981
RxPacketRate/s	TxByteRate/s	0.926
TxByteRate/s	RxByteRate/s	0.926
TxByteRate/s	Throughput/Kbps	0.926
TxPacketRate/s	RxByteRate/s	0.924
TxPacketRate/s	Throughput/Kbps	0.924
TxPacketRate/s	RxPacketRate/s	0.924
RxPackets	TxBytes	0.903
TxBytes	RxBytes	0.882
TxPackets	RxPackets	0.806
TxPackets	RxBytes	0.775
TxPackets	LostPackets	0.717

Observacoes da correlacao:

- TxPackets com TxBytes e RxPackets com RxBytes tem correlacao muito alta, o que e esperado por serem medidas do mesmo fluxo em unidades diferentes.
- TxPacketRate com TxByteRate tambem sao bastante correlacionados.
- Decisao: manter todos os pares correlacionados. Os modelos de arvore (Random Forest, XGBoost, LightGBM) sao robustos a multicolinearidade, e remover features poderia prejudicar a separabilidade.

7. Identificação de Problemas de Qualidade

Verifica os valores ausentes em todas as colunas com `df.isnull().sum()`. Complementa o `df.info()` da Seção 1, que mostra os non-null counts mas nao deixa explicito quais colunas tem problema. Faz sentido checar antes de usar imputacao no pipeline, ja que `SimpleImputer` so e util se houver algo a imputar.

```
# 7.1 Valores Ausentes
print("=== Valores Ausentes ===")
nulos = df.isnull().sum()
print(nulos.to_string())
print(f"Total de NaN no DataFrame: {df.isnull().sum().sum()}")
print(f"Colunas com pelo menos 1 NaN: {(nulos > 0).sum()}")
```

=== Valores Ausentes ===

```
FlowID          0
FlowDuration/s  0
SrcAddr         0
SrcPort         0
DstAddr         0
DstPort         0
Protocol        0
TxPackets       0
RxPackets       0
LostPackets     0
TxBytes         0
RxBytes         0
TxPacketRate/s  0
RxPacketRate/s  0
TxByteRate/s    0
RxByteRate/s    0
MeanDelay/s     0
MeanJitter/s    0
Throughput/Kbps  0
MeanPacketSize  0
PacketDropRate  0
AverageHopCount 0
label           0
Total de NaN no DataFrame: 0
Colunas com pelo menos 1 NaN: 0
```

Detecta registros identicos que poderiam vaziar informacao entre treino e teste com

`df.duplicated().sum()` . Tambem testo uma variante sem o FlowID, para capturar duplicatas de conteudo que tem IDs diferentes. Isso ajuda a entender se o split estratificado corre risco de contaminacao.

7.2 Duplicatas

```
n_dup = df.duplicated().sum()
n_dup_sem_id = df.drop(columns=['FlowID']).duplicated().sum()
print('=== Duplicatas ===')
print(f'Linhas completamente duplicadas (com FlowID): {n_dup}')
print(f'Linhas duplicadas ignorando FlowID: {n_dup_sem_id} ({n_dup_sem_id/len(df)*100:.1f}%)')
if n_dup_sem_id > 0:
    print('\nDistribuição das duplicatas por classe:')
    dup_mask = df.drop(columns=['FlowID']).duplicated(keep=False)
    print(df[dup_mask]['label'].value_counts().to_string())
    print('\nExemplo de duplicatas (sem FlowID):')
    print(df.drop(columns=['FlowID'])[df.drop(columns=['FlowID']).duplicated(keep=False)].head())
```

=== Duplicatas ===

Linhas completamente duplicadas (com FlowID): 0

Linhas duplicadas ignorando FlowID: 8205 (6.7%)

Distribuição das duplicatas por classe:

label

Flooding Attack 10451

Sybil Attack 302

Exemplo de duplicatas (sem FlowID):

	FlowDuration/s	SrcAddr	SrcPort	DstAddr	DstPort	Protocol	\
2161	185.897	192.168.0.150	9	192.168.0.27	9	UDP	
2163	185.897	192.168.0.151	9	192.168.0.27	9	UDP	
2165	185.897	192.168.0.152	9	192.168.0.27	9	UDP	
2167	185.897	192.168.0.153	9	192.168.0.27	9	UDP	

	TxPackets	RxPackets	LostPackets	TxBytes	...	RxPacketRate/s	\
2161	189	3	186	14364	...	0.016138	
2163	189	3	186	14364	...	0.016138	
2165	189	3	186	14364	...	0.016138	
2167	189	3	186	14364	...	0.016138	

	TxByteRate/s	RxByteRate/s	MeanDelay/s	MeanJitter/s	Throughput/Kbps	\
2161	77.2686	1.22649	0.145112	0.140107	0.009812	
2163	77.2685	1.22648	0.145145	0.140107	0.009812	
2165	77.2685	1.22648	0.145178	0.140107	0.009812	
2167	77.2685	1.22648	0.145211	0.140107	0.009812	

	MeanPacketSize	PacketDropRate	AverageHopCount	label
2161	76	0.984127	1.33333	Sybil Attack
2163	76	0.984127	1.33333	Sybil Attack
2165	76	0.984127	1.33333	Sybil Attack
2167	76	0.984127	1.33333	Sybil Attack

[4 rows x 22 columns]

Garante a integridade dos dados procurando valores fora do dominio fisico da rede. Uso mascaras booleanas em condicoes especificas: `PacketDropRate > 1` (uma taxa de perda nao pode passar de 100%), `FlowDuration < 0` (impossivel) e `AverageHopCount = 0`. Esses checks importam porque o `isnull()` so pega valores ausentes, nao valores invalidos.

7.3 Valores impossiveis

```
print('=== Valores Impossiveis ===')
```

```
checks = {
    'FlowDuration/s < 0': (df['FlowDuration/s'] < 0).sum(),
    'PacketDropRate < 0': (df['PacketDropRate'] < 0).sum(),
    'PacketDropRate > 1': (df['PacketDropRate'] > 1).sum(),
    'TxPackets < 0': (df['TxPackets'] < 0).sum(),
    'RxPackets > TxPackets': (df['RxPackets'] > df['TxPackets']).sum(),
    'AverageHopCount <= 0': (df['AverageHopCount'] <= 0).sum(),
    'MeanDelay/s < 0': (df['MeanDelay/s'] < 0).sum(),
    'MeanJitter/s < 0': (df['MeanJitter/s'] < 0).sum(),
    'Throughput/Kbps < 0': (df['Throughput/Kbps'] < 0).sum(),
    'LostPackets < 0': (df['LostPackets'] < 0).sum(),
```

```

}

for check, count in checks.items():
    status = 'PROBLEMA' if count > 0 else 'OK'
    print(f' {status} {check}: {count}')

```

```

=== Valores Impossíveis ===
OK FlowDuration/s < 0: 0
OK PacketDropRate < 0: 0
PROBLEMA PacketDropRate > 1: 81
OK TxPackets < 0: 0
OK RxPackets > TxPackets: 0
PROBLEMA AverageHopCount <= 0: 38501
OK MeanDelay/s < 0: 0
OK MeanJitter/s < 0: 0
OK Throughput/Kbps < 0: 0
OK LostPackets < 0: 0

```

Quantifica os outliers de cada feature numerica para decidir sobre transformacoes. Uso o metodo IQR, com limites em $Q1 - 1,5IQR$ e $Q3 + 1,5IQR$. Preferi ele ao Z-score porque nao assume distribuicao normal, o que combina com features de rede bem assimetricas. O resultado, 81 registros com PacketDropRate maior que 1, orienta a decisao de aplicar clip nessa feature dentro do Pipeline.

```

# 7.4 Outliers via IQR - features numéricas
print('=== Outliers via IQR (1.5x) ===')
outlier_counts = {}

for feat in NUM_FEATURES:
    Q1 = df[feat].quantile(0.25)
    Q3 = df[feat].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    n_out = ((df[feat] < lower) | (df[feat] > upper)).sum()
    outlier_counts[feat] = n_out

out_df = pd.Series(outlier_counts).sort_values(ascending=False)
out_pct = (out_df / len(df) * 100).round(2)
print(pd.DataFrame({'Outliers (IQR)': out_df, '%': out_pct})
      .query('`Outliers (IQR)` > 0').to_string())

```

```

=== Outliers via IQR (1.5x) ===
                Outliers (IQR)      %
DstPort          30332    24.83
SrcPort          30332    24.83
MeanPacketSize   30332    24.83
RxBytes          22636    18.53
RxPackets        21955    17.97
TxBytes          21784    17.83
AverageHopCount  20969    17.16
TxPackets        19951    16.33
LostPackets      18098    14.81
MeanDelay/s      14048    11.50
MeanJitter/s     12770    10.45
TxPacketRate/s   8760     7.17
TxByteRate/s     8431     6.90
RxPacketRate/s   7652     6.26
RxByteRate/s     7335     6.00
Throughput/Kbps  7335     6.00
FlowDuration/s   897      0.73

```

8. Decisões de Limpeza

Decisoes de Limpeza e Pre-processamento

1. FlowID: remover. Identificador sequencial unico por registro, sem valor preditivo.
2. SrcAddr e DstAddr: remover. Sao enderecos de uma rede simulada (NS-3). Um modelo que aprende IPs especificos da simulacao nao generaliza para redes UAV reais, onde os enderecos mudam a cada missao via roteamento AODV dinamico.
3. Protocol: remover. Variancia zero, 100% UDP nos 122.171 registros. Feature constante nao contribui para nenhum modelo, e o OneHotEncoding geraria uma coluna constante.
4. Valores ausentes: nenhuma acao. O dataset nao tem NaN. O SimpleImputer entra no Pipeline so como protecao para dados novos em producao.
5. Duplicatas (8.205 registros, 6,7%): manter. Estao concentradas no Sybil Attack, que cria varias identidades com IPs diferentes mas perfil de trafego identico. Essa repeticao e a propria assinatura do ataque, entao remover apagaria o padrao que o modelo precisa aprender.
6. PacketDropRate maior que 1 (81 registros, 0,07%): clip em 1,0. E matematicamente impossivel para uma taxa, um artefato de arredondamento da simulacao NS-3. Aplicado via `FunctionTransformer` dentro do Pipeline.
7. AverageHopCount igual a 0 (38.501 registros, 31,5%): manter. Nao e erro. Fluxos em que os pacotes nunca chegam ao destino (Blackhole descartando, loops de Sybil) tem hopcount zero de forma legitima. Remover tiraria parte do sinal de ataque.

8. Outliers nas features de volume: manter com StandardScaler. Um TxPacketRate extremo no Flooding e o proprio ataque, nao ruido. O StandardScaler normaliza a escala sem descartar os valores extremos.
9. Desbalanceamento (Flooding com cerca de 16%): usar `class_weight='balanced'` em todos os modelos, para penalizar os erros na classe minoritaria de forma proporcional.

9. Insights da EDA: Achados que Guiam a Modelagem

Os cinco achados mais importantes da EDA, que vao ser usados na fase de modelagem.

Insights da EDA: 5 Achados que Guiam a Modelagem

1. Protocol e constante (100% UDP), entao sai da lista de features. Parecia util antes de olhar os dados, mas a EDA mostrou variancia zero.
2. Flooding e a classe minoritaria, entao vou usar `class_weight='balanced'` em todos os modelos. Ele representa cerca de 16% contra 21% das demais, e o F1-macro penaliza isso corretamente.
3. Blackhole e Wormhole tem sobreposicao geometrica, e aqui modelos nao-lineares obrigatorios. Isso aparece nos scatterplots e no PCA, e tambem nos numeros: a LogisticRegression chega a F1 de 0,82 nesse par. As features derivadas (`loss_ratio` , `tx_efficiency`) entram entre as mais importantes do Random Forest.
4. AverageHopCount igual a 0 em 31,5% dos registros e comportamento de ataque, nao erro. Fluxos onde os pacotes nunca chegam tem hopcount zero de forma legitima. Mantive e documentei como zero valido, e calculei `throughput_per_hop` com epsilon de 1,0 para evitar divisao por zero.
5. As duplicatas (6,7%) sao a assinatura do Sybil Attack, entao ficam. Sao varias identidades com IPs diferentes e metricas de trafego identicas. Remover apagaria o padrao central desse ataque.

10. Feature Engineering

Criei 3 features derivadas pensadas para aumentar a separabilidade do par Blackhole e Wormhole.

Justificativa de cada uma:

- `loss_ratio` (LostPackets / TxPackets): normaliza a perda de pacotes pelo volume transmitido. O Blackhole tem PacketDropRate alto e o Wormhole pode ter baixo, entao essa razao captura a eficiencia real de entrega.
- `tx_efficiency` (RxBytes / TxBytes): razao de bytes que chegam ao destino. O Blackhole descarta bytes, enquanto o Wormhole os roteia pelo tunel, deixando RxBytes proximo de TxBytes.

- `throughput_per_hop` (Throughput / AverageHopCount): throughput normalizado pelo numero de saltos. Usei epsilon de 1,0 (e nao 1e-9) porque 31,5% dos registros tem hopcount zero, e um epsilon pequeno faria o valor explodir.

Feature descartada apos analise:

- `jitter_delay_product` (MeanJitter vezes MeanDelay): ficou perto do fim do ranking de importancia, sem poder discriminativo relevante. Removida.

```
# Criar features derivadas
df_eng = df.copy()

# Evitar divisão por zero
# ATENÇÃO: eps=1.0 para throughput_per_hop porque 31.5% dos registros têm
# AverageHopCount = 0 (comportamento de ataque). Com eps=1e-9 o max seria ~10^12.
eps_small = 1e-9 # para features onde zero é raro
eps_hop = 1.0 # para AverageHopCount (zero é frequente e válido)

df_eng['loss_ratio'] = df_eng['LostPackets'] / (df_eng['TxPackets'] + eps_small)
df_eng['tx_efficiency'] = df_eng['RxBytes'] / (df_eng['TxBytes'] + eps_small)
df_eng['throughput_per_hop'] = df_eng['Throughput/Kbps'] / (df_eng['AverageHopCount'] + eps_hop)

# jitter_delay_product foi testada e ficou na posição 22/23 no feature importance
# (importância 0.0084) – sem poder discriminativo relevante. Removida.

ENGINEERED_FEATURES = ['loss_ratio', 'tx_efficiency', 'throughput_per_hop']

print('Features derivadas criadas:')
print(df_eng[ENGINEERED_FEATURES].describe().round(4).T.to_string())
```

Features derivadas criadas:

	count	mean	std	min	25%	50%	75%	max
loss_ratio	122171.0	0.3693	0.3095	0.0	0.0909	0.2857	0.6429	1.3333
tx_efficiency	122171.0	0.6244	0.3115	0.0	0.3342	0.6975	0.9091	1.0000
throughput_per_hop	122171.0	0.7914	35.8099	0.0	0.0065	0.0273	0.7398	7974.4000

Valida se as features criadas tem poder de separar as classes. Uso `sns.boxplot` por classe para comparar a mediana e a dispersao de `loss_ratio`, `tx_efficiency` e `throughput_per_hop` em cada ataque. Se houvesse sobreposicao total, a feature nao agregaria nada; uma separacao visivel indica que o feature engineering valeu a pena.

```
# Boxplots das features derivadas por classe – verificar separabilidade BH vs WH
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle('Features Derivadas por Classe - Separabilidade BH↔WH',
             fontsize=13, fontweight='bold')

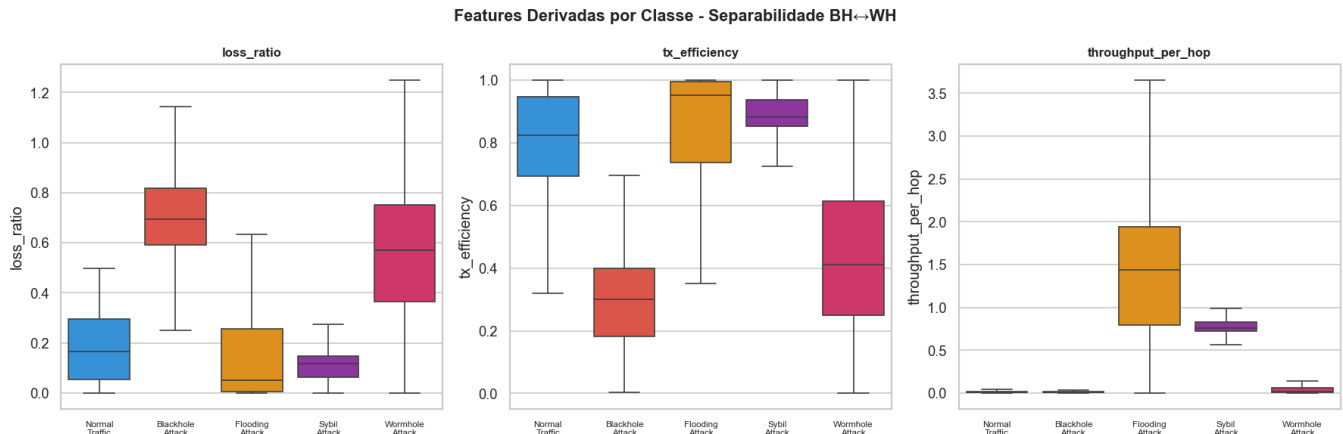
for ax, feat in zip(axes, ENGINEERED_FEATURES):
    sns.boxplot(
        data=df_eng, x='label', y=feat,
        order=ordem_classes,
        palette=PALETTE,
        ax=ax,
        showfliers=False
```

```

)
ax.set_title(feats, fontsize=10, fontweight='bold')
ax.set_xlabel('')
ax.set_xticklabels([c.replace(' ', '\n') for c in ordem_classes], fontsize=7)

plt.tight_layout()
plt.savefig('img/eda_img/eda_engineered_features.png', dpi=150, bbox_inches='tight')
plt.show()

```



O que o grafico das features derivadas por classe mostra:

- **loss_ratio:** Blackhole (mediana ~0,70, IQR de 0,59 a 0,82) e Wormhole (mediana ~0,58, IQR de 0,37 a 0,75) ficam bem acima das demais (Normal ~0,18, Flooding ~0,05, Sybil ~0,12). Separa bem os ataques que descartam pacotes.
- **tx_efficiency:** inverte o padrao. Normal, Flooding e Sybil ficam perto de 0,9 (alta eficiencia), o Blackhole cai para cerca de 0,30 (IQR de 0,18 a 0,40) e o Wormhole fica no meio, em torno de 0,42 (IQR de 0,25 a 0,60). E o sinal que melhor separa Blackhole de Wormhole.
- **throughput_per_hop:** o Flooding domina com mediana de cerca de 1,45 (IQR de 0,80 a 1,95), o Sybil fica em 0,80 e as demais perto de zero. Reforça a assinatura do Flooding.
- Resumindo: as 3 features derivadas adicionam separabilidade nos pontos certos. **loss_ratio** e **tx_efficiency** atacam justamente o par Blackhole versus Wormhole, e **throughput_per_hop** reforça o Flooding.

Mostra a separabilidade ganha com o feature engineering num scatterplot 2D colorido por classe. O par **tx_efficiency** e **loss_ratio** captura dois comportamentos opostos: o Blackhole com alta perda e baixa eficiencia, e o Flooding com alta transmissao e eficiencia distorcida. E a principal justificativa visual para incluir as features derivadas no pipeline.

```

# Scatterplot das features derivadas: tx_efficiency x loss_ratio, apenas BH vs WH
bh_wh_eng = df_eng[df_eng['label'].isin(['Blackhole Attack', 'Wormhole Attack'])].sample(
    n=min(10000, len(df_eng)), random_state=RANDOM_STATE
)

fig, axes = plt.subplots(1, 2, figsize=(12, 6))
fig.suptitle('Features Derivadas - Separabilidade BH<->WH vs Features Originais',
    fontsize=13, fontweight='bold')

```

```

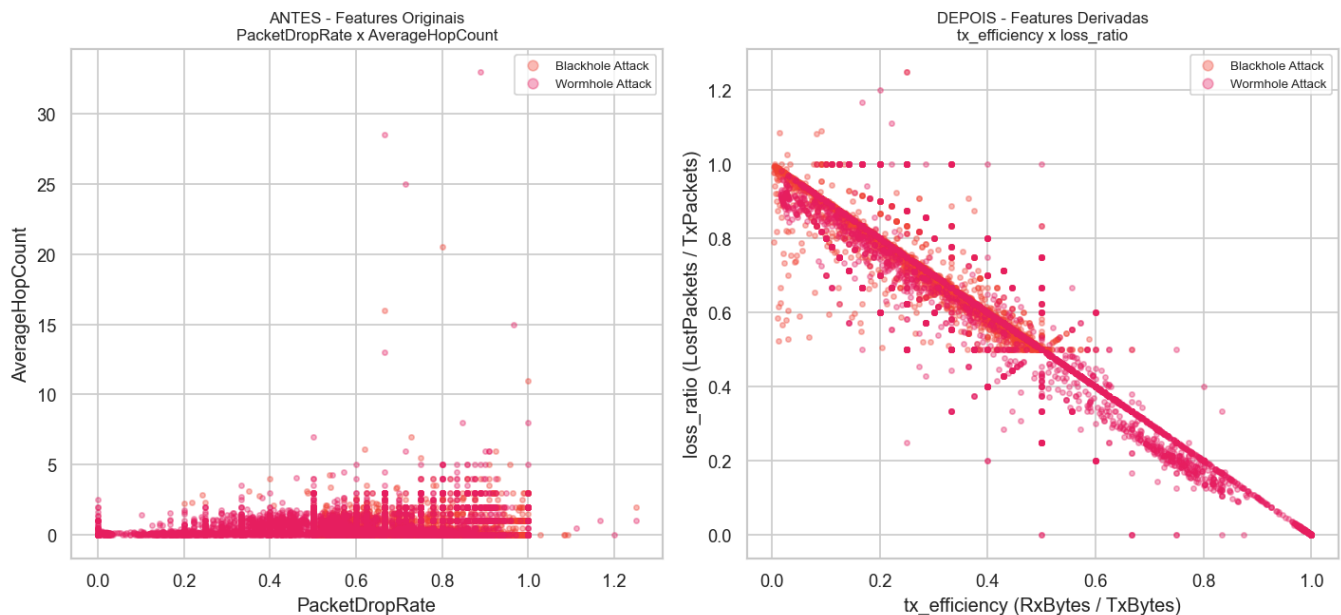
# Antes: PacketDropRate x AverageHopCount (features originais)
for cls in ['Blackhole Attack', 'Wormhole Attack']:
    sub = bh_wh_eng[bh_wh_eng['label'] == cls]
    axes[0].scatter(sub['PacketDropRate'], sub['AverageHopCount'],
                    c=PALETTE[cls], label=cls, alpha=0.35, s=10)
axes[0].set_title('ANTES - Features Originais\nPacketDropRate x AverageHopCount', fontsize=10)
axes[0].set_xlabel('PacketDropRate')
axes[0].set_ylabel('AverageHopCount')
axes[0].legend(fontsize=8, markerscale=2)

# Depois: tx_efficiency x loss_ratio (features derivadas)
for cls in ['Blackhole Attack', 'Wormhole Attack']:
    sub = bh_wh_eng[bh_wh_eng['label'] == cls]
    axes[1].scatter(sub['tx_efficiency'], sub['loss_ratio'],
                    c=PALETTE[cls], label=cls, alpha=0.35, s=10)
axes[1].set_title('DEPOIS - Features Derivadas\ntx_efficiency x loss_ratio', fontsize=10)
axes[1].set_xlabel('tx_efficiency (RxBytes / TxBytes)')
axes[1].set_ylabel('loss_ratio (LostPackets / TxPackets)')
axes[1].legend(fontsize=8, markerscale=2)

plt.tight_layout()
plt.show()

```

Features Derivadas - Separabilidade BH<->WH vs Features Originais



O que o grafico de separabilidade Blackhole versus Wormhole, antes e depois do feature engineering, mostra:

- No painel da esquerda (antes, PacketDropRate por AverageHopCount), os pontos de Blackhole e Wormhole se confundem na base do grafico, com hopcount baixo, por todo o range de PacketDropRate. Nao da para tracar uma fronteira.
- No painel da direita (depois, tx_efficiency por loss_ratio), os pontos formam uma estrutura quase linear inversa, ja que loss_ratio tende a 1 menos tx_efficiency por construcao. O Blackhole se acumula na regio de loss_ratio alto e tx_efficiency baixa (canto superior esquerdo), e o Wormhole cobre a faixa central e direita, com tx_efficiency mais alta. Da para ver uma separacao parcial.

- Resumindo: o feature engineering reduz a sobreposição entre os dois, principalmente quando o Blackhole derruba quase todos os pacotes (loss_ratio perto de 1). É a justificativa empírica para manter as features derivadas no pipeline.

11. Análise de Separabilidade via PCA Antes e Depois do Feature Engineering

```
# Preparar features para PCA
FEATURES_ORIGINAIS = [f for f in NUM_FEATURES] # só numéricas
FEATURES_COM_ENG = FEATURES_ORIGINAIS + ENGINEERED_FEATURES

# Subamostrar para agilidade visual
df_pca = df_eng.sample(n=min(20000, len(df_eng)), random_state=RANDOM_STATE)
labels_pca = df_pca['label'].values

# Features de volume com skew extremo (>50) dominam os componentes principais
# mesmo após StandardScaler, porque outliers extremos (Flooding) puxam a variância.
# Solução: aplicar log1p nas features de volume antes do PCA.
# Isso é APENAS para visualização – o Pipeline de modelagem usa StandardScaler direto.
VOLUME_LOG = [
    'TxPackets', 'RxPackets', 'LostPackets', 'TxBytes', 'RxBytes',
    'TxPacketRate/s', 'RxPacketRate/s', 'TxByteRate/s', 'RxByteRate/s',
    'Throughput/Kbps'
]

def preparar_pca(df_in, features):
    X = df_in[features].copy().fillna(0)
    for col in VOLUME_LOG:
        if col in X.columns:
            X[col] = np.log1p(X[col])
    return StandardScaler().fit_transform(X)

# PCA – features originais (com log1p nas de volume)
X_orig = preparar_pca(df_pca, FEATURES_ORIGINAIS)
pca_orig = PCA(n_components=2, random_state=RANDOM_STATE)
X_pca_orig = pca_orig.fit_transform(X_orig)

# PCA – features + derivadas (com log1p nas de volume)
X_eng = preparar_pca(df_pca, FEATURES_COM_ENG)
pca_eng = PCA(n_components=2, random_state=RANDOM_STATE)
X_pca_eng = pca_eng.fit_transform(X_eng)

print('Log1p aplicado nas features de volume para visualização PCA.')
print(f'Variância - originais: PC1={pca_orig.explained_variance_ratio_[0]:.1%}, PC2={pca_orig.explained_variance_ratio_[1]:.1%}, Total={pca_orig.explained_variance_ratio_.sum():.1%}')
print(f'Variância - depois da eg: PC1={pca_eng.explained_variance_ratio_[0]:.1%}, PC2={pca_eng.explained_variance_ratio_[1]:.1%}, Total={pca_eng.explained_variance_ratio_.sum():.1%}')
```

Log1p aplicado nas features de volume para visualização PCA.
 Variância - originais: PC1=44.3%, PC2=19.5%, Total=63.8%
 Variância - depois da eg: PC1=41.9%, PC2=17.0%, Total=58.9%

Compara a separabilidade global das 5 classes antes e depois do feature engineering, num espaço 2D.

Aplico `PCA(n_components=2)` nas features originais e depois nas features mais as derivadas. O PCA e

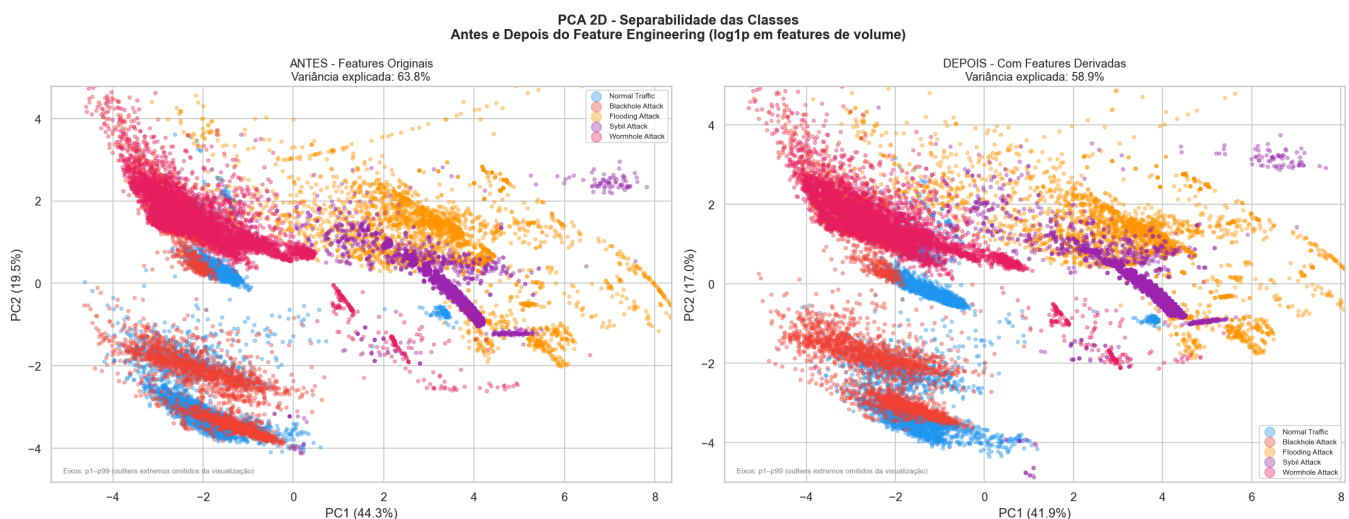
nao-supervisionado, entao mostra a separabilidade intrinseca dos dados sem usar o label. A variancia explicada pelos 2 componentes indica quanto da estrutura original e preservada na projecao.

```
fig, axes = plt.subplots(1, 2, figsize=(18, 7))
fig.suptitle('PCA 2D - Separabilidade das Classes\nAntes e Depois do Feature Engineering (log1p)',
             fontsize=13, fontweight='bold')

for ax, X_pca, pca_obj, titulo in [
    (axes[0], X_pca_orig, pca_orig, 'ANTES - Features Originais'),
    (axes[1], X_pca_eng, pca_eng, 'DEPOIS - Com Features Derivadas')
]:
    for cls in ordem_classes:
        mask = labels_pca == cls
        ax.scatter(
            X_pca[mask, 0], X_pca[mask, 1],
            c=PALETTE[cls], label=cls, alpha=0.35, s=10
        )
    # Limitar eixos ao percentil 1-99 para não deixar outliers extremos
    # esmagar o cluster principal num ponto
    p1_x, p99_x = np.percentile(X_pca[:, 0], [1, 99])
    p1_y, p99_y = np.percentile(X_pca[:, 1], [1, 99])
    margin_x = (p99_x - p1_x) * 0.15
    margin_y = (p99_y - p1_y) * 0.15
    ax.set_xlim(p1_x - margin_x, p99_x + margin_x)
    ax.set_ylim(p1_y - margin_y, p99_y + margin_y)

    var_total = sum(pca_obj.explained_variance_ratio_)
    ax.set_title(f'{titulo}\nVariancia explicada: {var_total:.1%}', fontsize=11)
    ax.set_xlabel(f'PC1 ({pca_obj.explained_variance_ratio_[0]:.1%})')
    ax.set_ylabel(f'PC2 ({pca_obj.explained_variance_ratio_[1]:.1%})')
    ax.legend(fontsize=7, markerscale=3, loc='best')
    ax.text(0.02, 0.02, 'Eixos: p1-p99 (outliers extremos omitidos da visualização)',
           transform=ax.transAxes, fontsize=7, color='gray', va='bottom')

plt.tight_layout()
plt.show()
```



O que o PCA 2D das 5 classes mostra:

- No painel da esquerda (antes, features originais, 63,9% de variancia explicada), aparecem 5 clusters parcialmente sobrepostos no centro. O Wormhole forma uma faixa horizontal no topo, o Blackhole fica logo abaixo, o Flooding a direita do centro, o Sybil em clusters distintos e o Normal ocupa a base e o centro-esquerda. Já da para ver alguma estrutura.
- No painel da direita (depois, com as features derivadas, 58,9% de variancia explicada), a estrutura geral se mantém, com novos bracos causados por loss_ratio e tx_efficiency. A variancia explicada cai, o que é esperado, porque há mais features para os 2 componentes resumirem, mas Blackhole e Wormhole ficam um pouco mais separáveis nos bracos inferiores.
- Resumindo: o PCA confirma que as 5 classes são parcialmente separáveis em 2D, mas os modelos não-lineares vão capturar muito mais estrutura no espaço completo.

Analisa isoladamente a fronteira mais difícil do dataset no espaço do PCA. Filtro o DataFrame para Blackhole e Wormhole e refaço a projeção PCA. O zoom isola o problema de separação mais crítico, a confusão de 10 a 27% citada no artigo, e ajuda a medir o ganho do feature engineering nesse caso específico.

```
# Zoom no par BH↔WH antes e depois
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('PCA 2D - Zoom: Blackhole vs Wormhole\nImpacto do Feature Engineering (eixos p1-p2)',
             fontsize=13, fontweight='bold')

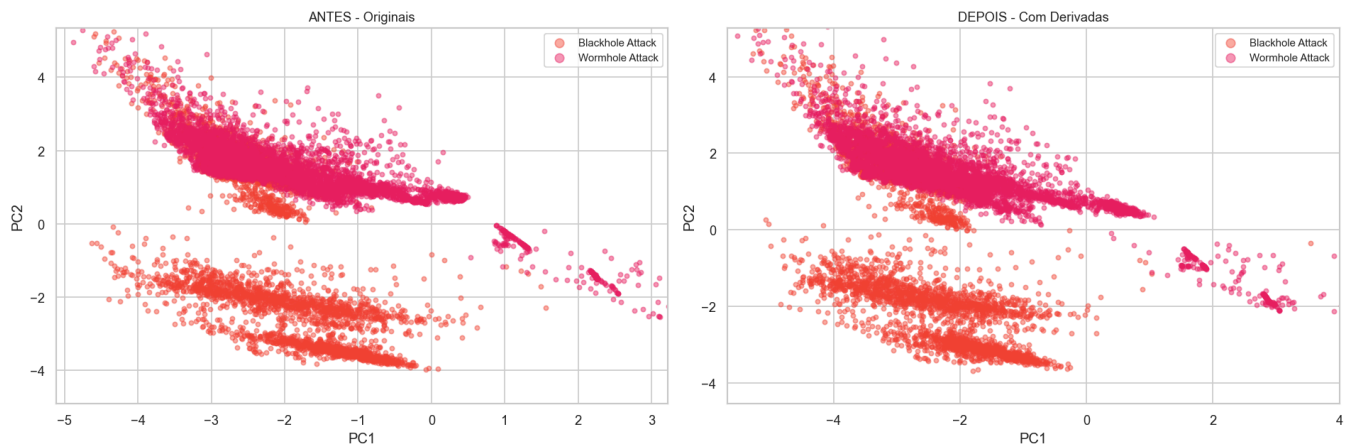
bh_wh_mask = np.isin(labels_pca, ['Blackhole Attack', 'Wormhole Attack'])

for ax, X_pca, titulo in [
    (axes[0], X_pca_orig, 'ANTES - Originais'),
    (axes[1], X_pca_eng, 'DEPOIS - Com Derivadas')
]:
    X_sub = X_pca[bh_wh_mask]
    for cls in ['Blackhole Attack', 'Wormhole Attack']:
        mask = (labels_pca == cls) & bh_wh_mask
        ax.scatter(
            X_pca[mask, 0], X_pca[mask, 1],
            c=PALETTE[cls], label=cls, alpha=0.45, s=14
        )
    # Limitar eixos ao p1-p99 do subconjunto BH+WH
    p1_x, p99_x = np.percentile(X_sub[:, 0], [1, 99])
    p1_y, p99_y = np.percentile(X_sub[:, 1], [1, 99])
    margin_x = (p99_x - p1_x) * 0.15
    margin_y = (p99_y - p1_y) * 0.15
    ax.set_xlim(p1_x - margin_x, p99_x + margin_x)
    ax.set_ylim(p1_y - margin_y, p99_y + margin_y)

    ax.set_title(titulo, fontsize=11)
    ax.set_xlabel('PC1')
    ax.set_ylabel('PC2')
    ax.legend(fontsize=9, markerscale=2)

plt.tight_layout()
plt.show()
```

PCA 2D - Zoom: Blackhole vs Wormhole
Impacto do Feature Engineering (eixos p1-p99)



- Nos dois painéis (antes e depois), as duas classes formam duas faixas diagonais sobrepostas, uma superior mais alongada e outra inferior mais compacta, cada uma com pontos das duas classes misturados.
- Comparando os painéis, a estrutura é bem parecida. O feature engineering não cria um corredor de separação claro em 2D, mas isso é esperado: o PCA é linear e a fronteira entre Blackhole e Wormhole costuma ser não-linear.

```
# Preparar dados completos
FEATURES_FINAIS = FEATURES_ORIGINAIS + ENGINEERED_FEATURES
# Adicionar Protocol codificado manualmente só para este experimento de análise
df_model = pd.get_dummies(df_eng[FEATURES_FINAIS + ['Protocol', 'label']],
                           columns=['Protocol'], drop_first=False)

feature_cols = [c for c in df_model.columns if c != 'label']
X_all = df_model[feature_cols].values
y_all = LabelEncoder().fit_transform(df_model['label'])

# RF simples para feature importance (não é o modelo final)
rf_eda = RandomForestClassifier(
    n_estimators=100,
    max_depth=15,
    class_weight='balanced',
    random_state=RANDOM_STATE,
    n_jobs=-1
)
rf_eda.fit(X_all, y_all)

importances = pd.Series(rf_eda.feature_importances_, index=feature_cols)
importances_sorted = importances.sort_values(ascending=False)

print('Top 20 features por importância:')
print(importances_sorted.head(20).round(4).to_string())
```

Top 20 features por importância:

TxByteRate/s	0.1337
TxPacketRate/s	0.1222
PacketDropRate	0.0757
RxByteRate/s	0.0729
loss_ratio	0.0640
FlowDuration/s	0.0576
throughput_per_hop	0.0575
tx_efficiency	0.0573
Throughput/Kbps	0.0568
RxPacketRate/s	0.0449
DstPort	0.0372
SrcPort	0.0348
LostPackets	0.0319
MeanPacketSize	0.0306
TxBytes	0.0247
TxPackets	0.0236
RxPackets	0.0179
RxBytes	0.0166
MeanDelay/s	0.0147
AverageHopCount	0.0142

Identifica quais features tem mais poder preditivo e valida a relevancia das derivadas, usando `RandomForestClassifier.feature_importances_` treinado no conjunto completo (sem split, so para esta analise exploratoria). Escolhi o Random Forest porque ele calcula a importancia pela reducao de impureza de Gini, que nao exige normalizacao e captura relacoes nao-lineares.

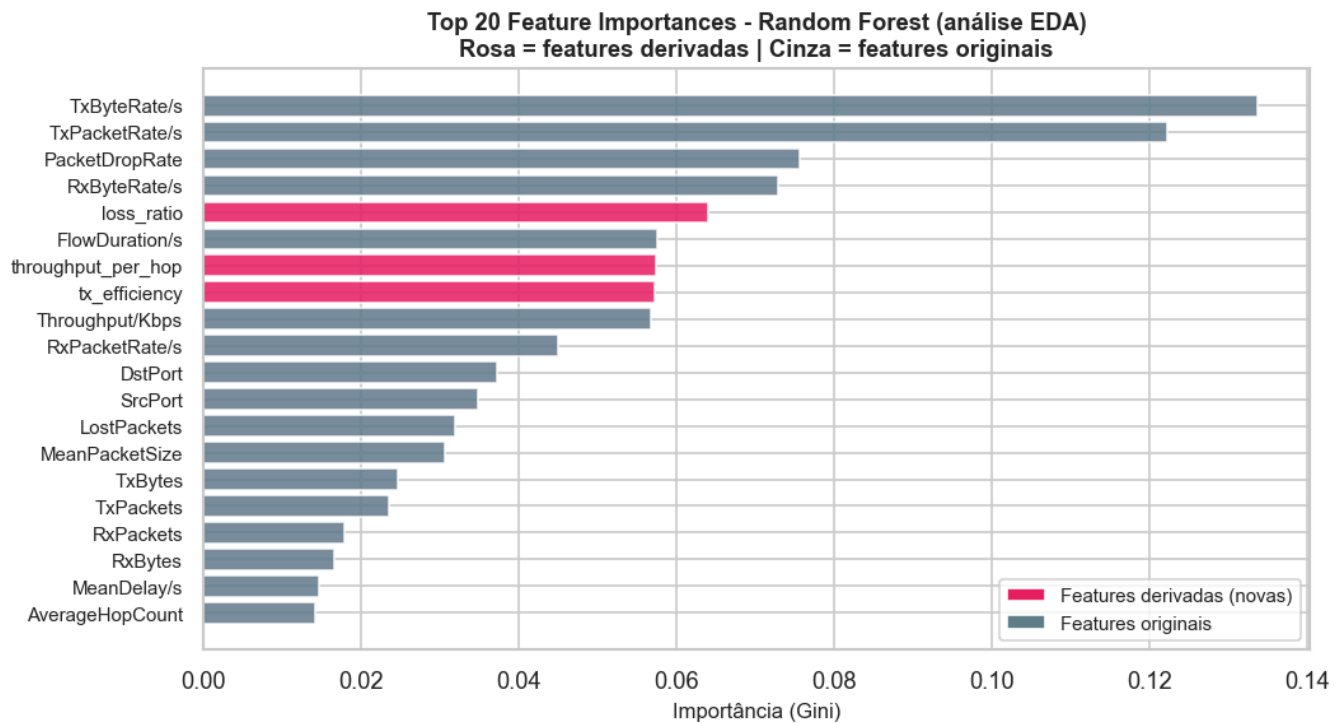
```
top_n = 20
top_features = importances_sorted.head(top_n)

# Colorir features derivadas diferente
cores = ['#E91E63' if f in ENGINEERED_FEATURES else '#607D8B' for f in top_features.index]

fig, ax = plt.subplots(figsize=(9, 5))
bars = ax.barh(range(len(top_features)), top_features.values[::-1],
               color=cores[::-1], alpha=0.85, edgecolor='white')
ax.set_yticks(range(len(top_features)))
ax.set_yticklabels(top_features.index[::-1], fontsize=9)
ax.set_xlabel('Importância (Gini)', fontsize=10)
ax.set_title(f'Top {top_n} Feature Importances - Random Forest (análise EDA)\n'
             'Rosa = features derivadas | Cinza = features originais',
             fontsize=11, fontweight='bold')

# Legenda manual
from matplotlib.patches import Patch
legend_elements = [
    Patch(facecolor='#E91E63', label='Features derivadas (novas)'),
    Patch(facecolor='#607D8B', label='Features originais')
]
ax.legend(handles=legend_elements, loc='lower right', fontsize=9)

plt.tight_layout()
plt.show()
```



O que o ranking de importancia (Top 20, Random Forest) mostra:

- As 4 primeiras são features de taxa originais: TxByteRate/s (~0,135), TxPacketRate/s (~0,122), PacketDropRate (~0,075) e RxByteRate/s (~0,072). A velocidade do tráfego e o sinal mais forte.
- A primeira feature derivada aparece em 5º lugar: `loss_ratio` (~0,063), batendo o Throughput/Kbps original. O feature engineering confirmou seu valor.
- As outras duas derivadas ficam no top 8: `throughput_per_hop` (7º, ~0,057) e `tx_efficiency` (8º, ~0,057). As três derivadas estão entre as 8 mais importantes de 20.
- As features de porta (DstPort e SrcPort) aparecem em 11º e 12º, com valor preditivo modesto, mas não desprezível.
- Resumindo: as features derivadas justificaram sua inclusão, em especial a `loss_ratio`, que resume numa única variável a assinatura central do Blackhole, que é descartar pacotes.

Confirma de forma quantitativa se `loss_ratio`, `tx_efficiency` e `throughput_per_hop` agregaram valor preditivo, consultando direto o ranking de importâncias. Se as derivadas aparecem no top 10, o feature engineering está validado; se ficassem abaixo do top 20, a inclusão precisaria de outra justificativa, como interpretabilidade de domínio.

```
# Verificar posição das features derivadas no ranking
print('Posição das features derivadas no ranking de importância:')
all_ranked = list(importances_sorted.index)
for feat in ENGINEERED_FEATURES:
    pos = all_ranked.index(feat) + 1
    imp = importances_sorted[feat]
    print(f' {feat:30s} -> posição {pos:3d} de {len(all_ranked)} | importância: {imp:.4f}')
```

Posição das features derivadas no ranking de importância:

loss_ratio	-> posição	5 de 22	importância: 0.0640
tx_efficiency	-> posição	8 de 22	importância: 0.0573
throughput_per_hop	-> posição	7 de 22	importância: 0.0575

Interpretacao dos Resultados da EDA

A analise exploratoria mostrou um dataset sintetico de boa qualidade: sem valores ausentes, sem tipos errados e com apenas 81 registros (0,07%) com PacketDropRate acima de 1, um artefato do simulador NS-3 que foi corrigido com clip. As 8.205 duplicatas (6,7%) foram mantidas porque representam a assinatura real do Sybil Attack, que cria identidades falsas com metricas identicas.

O que as features dizem sobre cada ataque

Cada ataque tem um comportamento proprio que aparece nas features de rede:

- Flooding gera trafego massivo, e o Throughput/Kbps e a feature que mais o separa das demais classes.
- Sybil perde muitos pacotes em volume absoluto (LostPackets alto) e gera registros duplicados.
- Blackhole descarta pacotes silenciosamente, com PacketDropRate alto (cerca de 0,70) e tx_efficiency baixa (cerca de 0,30).
- Wormhole cria tuneis que aumentam o delay e o jitter, com MeanDelay e AverageHopCount elevados.
- Normal Traffic fica no canto inferior esquerdo de quase todos os graficos: baixo delay, baixo jitter e baixa perda.

Flooding, Sybil e Normal sao facéis de separar. O problema esta no par Blackhole versus Wormhole.

O problema central: Blackhole versus Wormhole

Os dois ataques mexem no roteamento de formas diferentes, mas no nivel das metricas agregadas por fluxo as assinaturas se confundem. Os scatterplots (PacketDropRate por HopCount, MeanDelay por MeanJitter) mostram bastante sobreposicao entre as duas classes, e o PCA 2D reforca isso: mesmo depois do feature engineering, elas formam faixas diagonais sobrepostas no espaco projetado.

Isso tem duas consequencias diretas para a modelagem:

1. Modelos lineares nao dao conta. A fronteira entre Blackhole e Wormhole e nao-linear, entao um modelo como a LogisticRegression erra de forma sistematica nesse par. Precisamos de modelos de arvore (Random Forest, XGBoost, LightGBM).
2. O feature engineering e essencial. As 3 features derivadas (loss_ratio, tx_efficiency, throughput_per_hop) foram criadas justamente para ampliar a diferenca entre quem descarta pacotes (Blackhole) e quem reencaminha por tuneis (Wormhole). As tres ficaram no top 8 de importancia do Random Forest, o que valida a decisao.

O que esperar na modelagem

Com base na EDA, a expectativa e:

- F1-macro geral acima de 0,95, puxado pelas 3 classes faceis (Normal, Flooding e Sybil).
- F1 individual de Blackhole e Wormhole bem mais baixo, em torno de 0,90, sendo a principal fonte de erro.
- A matriz de confusao com o erro concentrado no par Blackhole versus Wormhole, com o modelo trocando um pelo outro.
- Experimentos de label engineering agrupando Blackhole e Wormhole devem reduzir bastante o erro.

Essas hipoteses sao testadas e validadas no notebook de modelagem (`02_modelagem.ipynb`).